

**Part 0. Before you begin**

Required materials:

- a sample solution file to be demonstrated to students.
- A4 sheets (1 sheet per student).

Teaching materials:

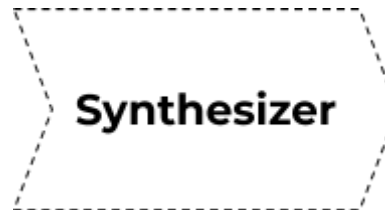
- ❑ Link to the test to pick up the appropriate projects.
- ❑ [Link to the sample solution.](#)
- ❑ [Link to the slides containing project implementation stages.](#)

Recommended implementation time for the project:

Two lessons.

**Part 1. General project information***Minimal prior knowledge required from students*

- Each student :
- knows how to create sprites of any type in Play;
 - understands that additional files used in a project must be added to the project folder;
 - understands how to work with sounds in Pygame (knows the methods for background music and sound);
 - knows how to create lists with different data types;
 - applies clear() and append() methods to the created lists
 - has successfully implemented the "Piano" learning project and wants to develop it further.

*Learning results*

- Student has created the project check list.
- Student put into practice his/her knowledge of lists (used lists to consistently store different kinds of data -> created a list of note lists).
- Student used Play to create a new type of UI elements — musical instrument selectors.

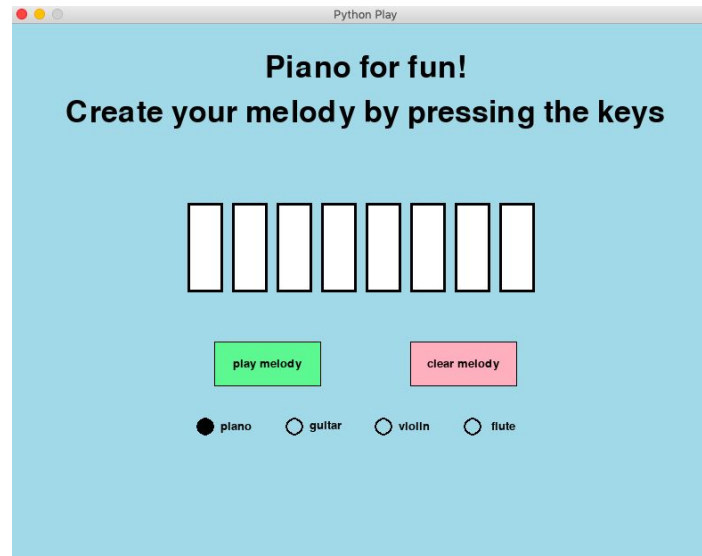
Ideas:

- Student has understood that Python is used to program software in addition to games (?)
- Each student got an interesting and visually appealing project, which would be cool to play after class.



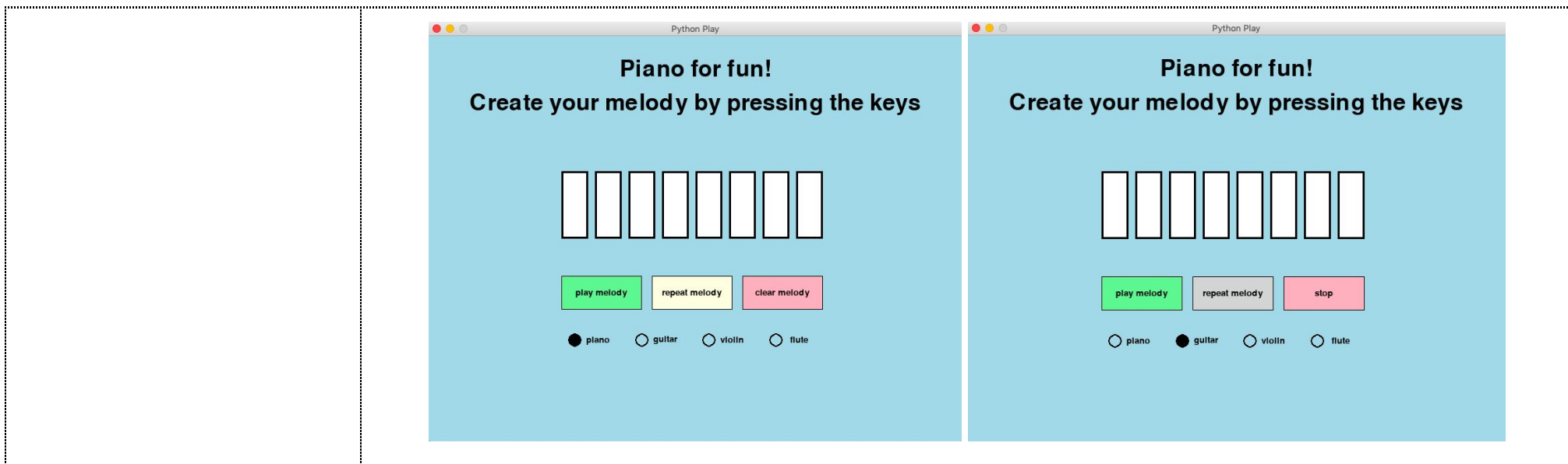
Part 2. Description of the expected results

<i>Program</i>	<i>Features</i>
Synthesizer	• Welcomes user with hints and plays back hello.mp3 music file on startup. • If user presses keys, Synthesizer records them as a melody. When a key is clicked, the respective note is played (piano sound is always used). • Has "Play melody" button which plays back a recorded melody. If the button is clicked immediately after the program starts, no playback is performed. • Has "Clear melody" button which deletes a recorded melody. • Has a selector for each musical instrument (four in total: piano, guitar, cello, flute). A single musical instrument can be selected at any time. Melody is also played back using "Play melody" button.



- *Optionally:* Play back a recorded melody on the selected musical instrument. To do this, press "Repeat melody" button (melody will be played back until user stops it manually).

When "Repeat melody" is pressed, "Clear melody" changes to "Stop". When user presses "Stop", they interrupt looping melody playback, while "Stop" changes back to "Clear melody".



Part 3. Project stages

IMPORTANT NOTICE.

Before picking up the projects, make sure to suggest the test for the students ([link](#)). To run the test, each student need to launch a browser and sign in to the platform, since AlgoVSCode does not support this test. The test includes basic theoretical questions about the topics studied in the project module and checks the corresponding practical skills of the students. Its results are for information only and should help the teacher understand whether the students are ready for harder tasks.

For students who made **less than three mistakes**, we recommend trying out "Synthesizer" or "Arkanoid". For students who made **three or more mistakes**, we recommend getting back to the previous elective projects and picking up any of remaining.

However, the test cannot clearly figure out whether a particular student is ready for harder tasks or not. **Teacher will make the final choice between "Synthesizer" or "Arkanoid" only after a discussion with the student.**

The goal of the project is to improve understanding of lists and finalize knowledge of the Play library. This project follows organically from the "Piano" learning project. By extending "Piano" to "Synthesizer", student arrives at the idea that a brand new concept can be developed from a familiar project.

In this project, creating a mind map is not required. Students have already implemented a basic "Piano" learning project and explored its structure in detail. They have a ready-to-use foundation for "Synthesizer".

"Synthesizer" calls for more advanced application of language constructs that students already know. For example, sounds[] is a list of lists which can be hard to deal with for a novice student. More advanced students may be offered to implement program states (e.g., instrument selection) using Game class and its methods. It all calls for teacher's attention and joining forces of students who have chosen "Synthesizer".

#	Stages and their goals	Content	Format	Results
1	Readiness test "Play, lists, and for" Pass the test on the platform. Count the mistakes, discuss them with the teacher and make the decision on whether to proceed to harder projects or not.	Start the lesson by introducing the pool of available projects. Remind students the "rules of the game": they have to pass a small test during this lesson to prove that they are ready for harder projects. Underline that the purpose of the test is not to split the group into smart and stupid. Say that it is very important to understand labor intensity of each "serious" project. If you pick up a task which is too labor-consuming, chances are that you will fail. Therefore you should always choose tasks of adequate difficulty and elaborate them without any hassle. Since we do not have much experience in selecting projects, let's see where we are at by doing a test. Conduct the test for each student. Do not introduce any time limits, but keep an eye on the students to make sure they do not help each other. Once you notice that a student has completed the	Individual work on the platform. Discussion of the results with the teacher.	Each student now understands his/her actual knowledge level thanks to the teacher and is ready to pick up a project.

		test, ask him/her to come to your table (so that other students did not pay too much attention to your presence) and discuss the results. Make the decision on whether to proceed to harder projects or not. Remember that the teacher's opinion must prevail over all other factors. Pay special attention to students who made many mistakes. These are students for whom you need to decide whether they are ready for the next projects. For students who have successfully passed the test, suggest getting familiar with "Synthesizer: FS" and "Arkanoid: FS" and picking up one of them. Note. Even if the student has completely failed the test, make sure to emphasize that you will elaborate all the mistakes with him/her before getting started with the project. If we resolve all the issues now, we will prevent such mistakes in future.		
2	"Projects fair" Learn or repeat the project stages. Select a project from the pool of available projects. Get a functional specification (FS). Review the requirements and get started with the project.	If you and the student decided to keep the current difficulty level, then proceed to the training manuals for "Screen pet" and "Gaming machine". Before picking up the projects, make sure to briefly remind the students of the project stages. Remember that revision will be effective if you only ask questions and prompt some of the answers, while the students are revising the stages by themselves.	Work with the teacher.	Each student has picked up a project. Each student has received and reviewed an FS, and is ready to get started.

		Project stages 📌 Get familiar with the idea/Generate an idea 🔧 Prepare the tools (images, guidelines) 📋 Split the projects into subtasks -> Project scheme 🔄 Iterative work on the project: program -> review (test) -> finalize 🚀 Release the project to production  Once the revision is over, everyone may choose a project for work. Then the students should sign in to the platform and review the functional specifications for the projects ("Synthesizer: FS" or "Arkanoid: FS"). Closely watch the project selection. Keep in mind that "Synthesizer" should be approved for the kids who have successfully completed "Piano". If that learning project was too difficult, choosing "Arkanoid" is preferable. After review, students should get started with the first task — create a project checklist.		
--	--	--	--	--

The description provided below is only related to "Synthesizer" project stages

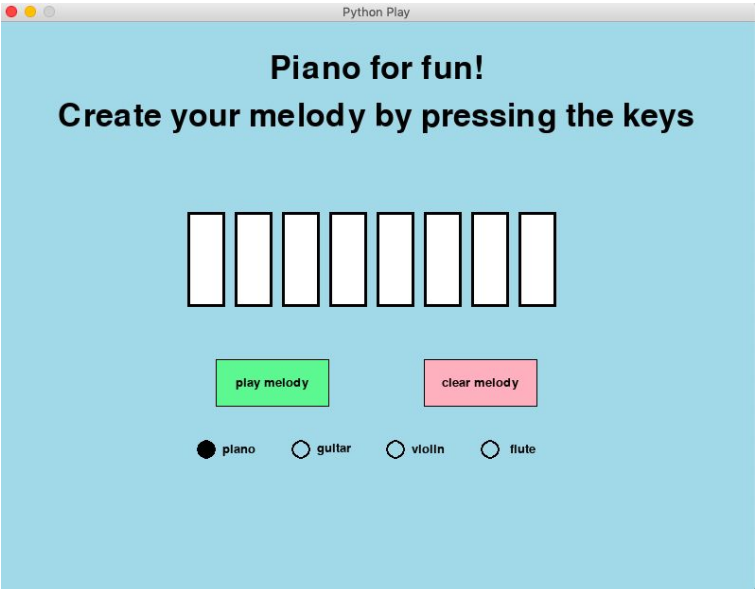
3	Splitting a project into tasks Sign in to the platform. Locate the	Arrange the creation of checklists by the students. Since they already know what a typical checklist should look like, give them more freedom to do their work on	Individual work, work with the teacher.	Students split their projects into tasks.
---	---	--	--	--

second level of "Synthesizer: FS"(where the requirements to a checklist are shown). Assess whether familiar tools and theory are sufficient to complete the project (yes). Create a project checklist on an A4 sheet on your own or together with the teacher.	their own. The goal of each student is to describe his/her consecutive actions for the project implementation. Each student must create a checklist on their own or in collaboration with another student who has also picked up "Synthesizer". There are different ways to do the decomposition. This is one of the options made up as a list of extensions for "Piano" project: 1. Create selector sprites to choose a musical instrument and auxiliary sprites for selectors (keypress visual design, as for a piano key). 2. Rearrange sounds[] list to store sounds of multiple musical instruments (list of lists). 3. Implement recording a melody as pressed key numbers and clearing a recorded melody. 4. Implement playing back a melody using a selected musical instrument. Recall that the more difficult it is for a student to program, the more detailed work plan is needed.		The tasks are arranged and added to the checklist. A sample mind map for a project is shown in part 4 "Sample project implementation".
--	---	--	--

Note. If it is hard for a student to compose a checklist, then suggest making a mind map. A block diagram will not be suitable in this case, because "Arkanoid" has a complex structure. Try not to leave the student face to face with problems. Whenever possible, sit down nearby and discuss

the mind map together. Or try to find a pair for the student and organize their teamwork. A possible mind map for the project is provided in the next section.

4	Creating selector sprites Synthesizer first startup Copy "Piano" code into the file containing "Synthesizer" code. Explore sound files in the project folder. Create sprites for missing UI elements. Launch the project, evaluate its visual appearance and eliminate artifacts.	Arrange individual work. Student has all required knowledge to enhance "Piano" interface. If necessary, discuss visual appearance of a selector press with the student. Open your project (sample solution) and demonstrate selectors. Ask the student how a piano key press is implemented (sprite color changes for 0.3 seconds). Use questions to lead the student to a similar solution: ❑ change the color of selector sprite itself (darken) and other selectors (brighten) when selector sprite is pressed.  Upon completion of this stage, "Piano" project should transform into "Synthesizer" and gain the following UI elements: <pre># selector for piano ch_p = play.new_circle(color='black', x=-180, y=-150, radius=10, border_color='black', border_width=2) txt_p = play.new_text(words='piano', x=-145, y=-150, font_size=20)</pre>	Individual work.	"Piano" program with new UI elements. In the window, new sprites are displayed which are still inactive.
---	--	---	-------------------------	--

		At startup, the application will look like this:  Buttons and selectors are still inactive.		
--	--	---	--	--

Though all the theoretical topics for the next stage are already known to students, using this knowledge in complex cases may require some assistance from the teacher.

5	Arrange loading and storing sounds of four musical instruments.	An important part of the project is to arrange storage of sounds of various musical instruments. The sounds are already in the project folder. • 1.ogg - 8.ogg piano (default) • f1.ogg - f8.ogg flute • g1.ogg - g8.ogg guitar • v1.ogg - v8.ogg cello • hello.mp3 - welcome music file	Individual work, work in pairs, work with the teacher.	The program loads all sounds from the project folder and is ready to play them. Student has realized that lists can store elements of any type,
---	---	--	---	---

- clear_melody.wav - melody deletion sound.

Previously in the "Piano" project, sounds were stored in sounds[] list. Offer a student to arrange storage in the same list. Remind him/her that a list may contain anything, including other lists.

If student finds it hard to do the task, divide it into smaller parts and suggest to do them one at a time:

Task 1. Create four empty sound lists in sounds[].

Solution. Create an empty sound[] list and populate it with four empty lists:

```
sounds = []  
# we have 4 instruments:  
for i in range(4):  
    sounds.append([])
```

Task 2. Populate sounds[] with loaded sounds.

Solution. Extend the loop from "Piano" which draws 8 keys and loads the sounds. The loop is executed 8 times. At each iteration, draw a key and load piano sound into sounds[0], guitar sound into sounds[1], etc.

Note that sound names are formed directly in the loop by concatenating strings. When i=1, the command

```
sound = pygame.mixer.Sound('g' + str(i+1) + '.ogg')
```

load sound g1.ogg from the project folder into the program.
Possible solution of this stage can look like this:

including large
sound lists.

		<pre> for i in range(8): key_x = -180 + i * 50 # 40 - key width, 10 - gap between keys key = play.new_box(color='white', border_color='black', border_width=3, x=key_x, y=50, width=40, height=100) keys.append(key) # all sounds of all instruments are stored in a single list sound = pygame.mixer.Sound(str(i+1) + '.ogg') sounds[0].append(sound) sound = pygame.mixer.Sound('g' + str(i+1) + '.ogg') sounds[1].append(sound) sound = pygame.mixer.Sound('v' + str(i+1) + '.ogg') sounds[2].append(sound) sound = pygame.mixer.Sound('f' + str(i+1) + '.ogg') sounds[3].append(sound) </pre>		
--	--	---	--	--

Technical note for teacher. Note that elements of sounds[] list are four lists. When accessing sounds[0] element, program returns a list containing piano sounds. Similarly, sounds[1] contains sounds, etc.

Internally, sounds list looks like this: **[[], [], [], []]**.

For comparison:

- upon sounds.append(sound) command, sounds list will look like this: **[[], [], [], [], sound]**
- upon sounds[0].append(sound) command, sounds list will look like this: **[sound], [], [], []**.

The second option must be used to store sounds of all instruments in a single list.

6	Recording a melody as pressed key numbers, playing back and clearing a melody Finalize @play.when_program_starts and @play.repeat_forever sections of "Piano" project.	Now "Piano" needs to be technically enhanced considering new features in the previous item. Arrange individual work. You have already done all complex preparations, so students should complete the work without assistance (or at least try). In case of difficulties, try answering with leading questions and pair up the students. The only thing you can immediately help a student with is	Individual work, work in pairs.	Program can memorize a melody played on keys for the selected musical instrument (however, selectors are not set up yet).
---	---	--	------------------------------------	---

Finalize @key_play_melody.when_clicked (playback) and @key_clear_melody.when_clicked (melody deletion).	creating instrument global variable. Student must look at the sections of the finalized "Piano" program and determine if there is anything to be changed. Student's expected conclusions: ❑ @play.when_program_starts and @key_clear_melody.when_clicked should not be changed. ❑ @key_play_melody.when_clicked section needs to be changed, because melody must be played on the selected instrument. Problem: where and how can number of the currently selected instrument be obtained? Solution: create a global variable holding the number of currently selected instrument. <pre># piano by default instrument = 0</pre> Changing values of this variable will be described in programming selectors at the next stage. Then the loop body needs to be modified. Now, instead of sounds[], we access the list it contains. In the string below, we kind of say: "Play back sound i from melody list on selected instrument":		
--	--	--	--

		<pre>sounds[instrument][melody[i]].play() # play a note of required instrument ("list of lists")</pre> ❑ In @play.repeat_forever section, loop body also needs to be changed. If we want to always hear piano when pressing synthesizer keys, a possible solution looks like this: <pre>sounds[0][i].play() # let keys always sound like piano</pre> If a student also wants synthesizer to play a selected instrument, change its index to instrument similarly to the previous item.		
--	--	---	--	--

Technical note. Notice different use cases for the global variable. By now, we have just initialized instrument variable, but have not changed its value (if fact, we used it as a constant). We shall separately discuss an instance when we decide to change its value locally (for example, inside a function). Consider a part of the program playing back a recorded melody. Imagine we have no selectors and always have to play back the melody on guitar:

```
@key_play_melody.when_clicked
# play back a recorded melody on guitar
async def play_m():
    global instrument # program needs to be told that it is exactly that global variable
    instrument = 1 # a value can now be assigned without a chance of confusion
    for i in range(len(melody)):
        await play.timer(seconds=0.5)
        sounds[instrument][melody[i]].play()
```

Program needs to be told that we are going to work exactly with that global variable. Otherwise, instrument = 1 assignment will lead to a confusion: are we changing a local or a global variable? For this reason, we need to specify that we are working exactly with a global variable.

A global variable is not the only way to store an instrument number. Another method is provided in additional tasks.

7	Setting up instrument selectors Create sections (one per each musical instrument) for instrument selectors. Each selector function must: - set the required musical instrument - visualize a selector press (see stage 1).	Before you explore this item, read the technical note above. Now it is time to handle selector presses. Student has all required knowledge to implement this stage. If students have difficulties setting up the first selector, pair them up and reduce your help to leading questions. Handling a selector sprite press is done similarly to previous projects. A possible solution for a guitar selector looks like this: ❑ Create a section for handling a selector press event: <pre>@ch_g.when_clicked def set_guitar(): pass</pre> ❑ In the function, we tell the program that we are going to change value of the global variable, and set guitar as the instrument in the next string: <pre>global instrument instrument = 1</pre> ❑ Then we change appearance of the selectors. We darken the guitar selector and brighten others: <pre>ch_p.color = 'light blue' ch_g.color = 'black'</pre>	Individual work, work in pairs, work with the teacher.	The basic part of the project is complete. If there is enough time left, the student may proceed to additional tasks.
---	--	---	---	---

		<pre>ch_v.color = 'light blue' ch_f.color = 'light blue'</pre> Other selector presses are handled in a similar way. Complete solution is provided in Part 4: "Sample solution".		
--	--	--	--	--

Now the project can be delivered to the customer.

8	Project delivery (testing by teacher)	Once a student says that the project is completed, arrange its testing. It is performed in three stages: - Testing by the developer. The student must open the project description and check if the required actions have been performed properly. - Peer review and testing by the students. To be performed similarly to code review. - Testing by the teacher. Check the program. If everything works fine, ask the student questions to understand his/her knowledge of the topic. If the student answers them successfully, offer him/her additional tasks.	Individual work, work in pairs, work with the teacher.	The basic part of the project is complete. If there is enough time left, the student may proceed to additional tasks.
---	--	---	--	---



Part 4. Sample project implementation

Project checklist:

1. Create selector sprites to choose a musical instrument and auxiliary sprites for selectors (keypress visual design, as for a piano key).
2. Rearrange sounds[] list to store sounds of multiple musical instruments (list of lists).
3. Implement recording a melody as pressed key numbers and clearing a recorded melody.

4. Implement playing back a melody using a selected musical instrument.

Code

```
import pygame
import play

instrument = 0

# interface - hints, control buttons:
play.set_backdrop('light blue')
introduce1 = play.new_text(words='Piano for fun!', x=0, y=250)
introduce2 = play.new_text(words='Create your melody by pressing the keys', x=0, y=200)

key_play_melody = play.new_box(color='light green', border_color='black', border_width=1, x=-110, y=-80, width=120, height=50)
kpm = play.new_text(words='play melody', x=-110, y=-80, font_size=20)

key_clear_melody = play.new_box(color='light pink', border_color='black', border_width=1, x=110, y=-80, width=120, height=50)
kcm = play.new_text(words='clear melody', x=110, y=-80, font_size=20)
sound_clear_melody = pygame.mixer.Sound('clear_melody.wav')

# instrument selectors:
ch_p = play.new_circle(color='black', x=-180, y=-150, radius=10, border_color='black', border_width=2)
txt_p = play.new_text(words='piano', x=-145, y=-150, font_size=20)
ch_g = play.new_circle(color='light blue', x=-80, y=-150, radius=10, border_color='black', border_width=2)
txt_g = play.new_text(words='guitar', x=-45, y=-150, font_size=20)
ch_v = play.new_circle(color='light blue', x=20, y=-150, radius=10, border_color='black', border_width=2)
txt_v = play.new_text(words='violin', x=55, y=-150, font_size=20)
ch_f = play.new_circle(color='light blue', x=120, y=-150, radius=10, border_color='black', border_width=2)
txt_f = play.new_text(words='flute', x=155, y=-150, font_size=20)

# keys and their sounds:
keys = []
sounds = []
```

```

# we have 4 instruments:
for i in range(4):
    sounds.append([])

for i in range(8):
    key_x = -180 + i * 50 # 40 - key width, 10 - gap between keys
    key = play.new_box(color='white', border_color='black', border_width=3, x=key_x, y=50, width=40, height=100)
    keys.append(key)
    # all sounds of all instruments are stored in a single list
    sound = pygame.mixer.Sound(str(i+1) + '.ogg')
    sounds[0].append(sound)
    sound = pygame.mixer.Sound('g' + str(i+1) + '.ogg')
    sounds[1].append(sound)
    sound = pygame.mixer.Sound('v' + str(i+1) + '.ogg')
    sounds[2].append(sound)
    sound = pygame.mixer.Sound('f' + str(i+1) + '.ogg')
    sounds[3].append(sound)

# melody not yet recorded
melody = []

@play.when_program_starts
async def start():
    pygame.mixer_music.load('hello.mp3')
    pygame.mixer_music.play()
    await play.timer(seconds=4.5)

@ch_p.when_clicked
def set_piano():
    global instrument
    instrument = 0
    ch_p.color = 'black'
    ch_g.color = 'light blue'
    ch_v.color = 'light blue'

```

```
    ch_f.color = 'light blue'

@ch_g.when_clicked
def set_guitar():
    global instrument
    instrument = 1
    ch_p.color = 'light blue'
    ch_g.color = 'black'
    ch_v.color = 'light blue'
    ch_f.color = 'light blue'

@ch_v.when_clicked
def set_violin():
    global instrument
    instrument = 2
    ch_p.color = 'light blue'
    ch_g.color = 'light blue'
    ch_v.color = 'black'
    ch_f.color = 'light blue'

@ch_f.when_clicked
def set_flute():
    global instrument
    instrument = 3
    ch_p.color = 'light blue'
    ch_g.color = 'light blue'
    ch_v.color = 'light blue'
    ch_f.color = 'black'

@key_clear_melody.when_clicked
def clear():
    melody.clear()
    sound_clear_melody.play()
```

```

@key_play_melody.when_clicked
# play back a recorded melody
async def play_m():
    for i in range(len(melody)):
        await play.timer(seconds=0.5)
        sounds[instrument][melody[i]].play() # play a note of required instrument ("list of lists")

@play.repeat_forever
async def play_piano():
    for i in range(len(keys)):
        if keys[i].is_clicked:
            keys[i].color = 'light grey'
            sounds[0][i].play() # let keys always sound like piano
            await play.timer(seconds=0.3)
            keys[i].color = 'white'
            melody.append(i)
play.start_program()

```

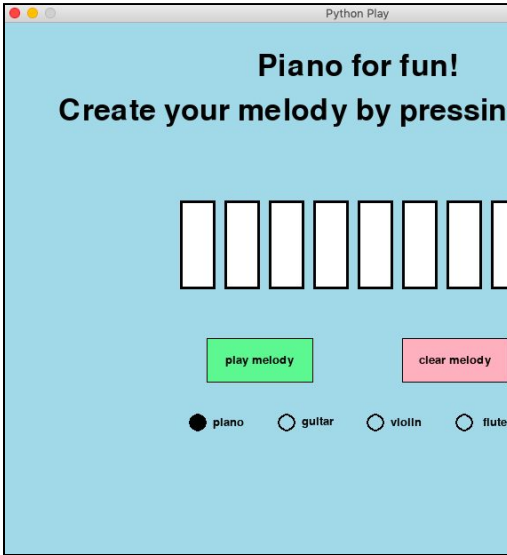


Part 5. Methods for evaluating the results

Stage 1. Program operability check

Any program must handle the following test cases. If all the tests are passed, this means that the student has achieved the mandatory basic level of knowledge.

Program testing	
The program is launched, no keys have been pressed yet.	Program plays back a welcome melody. Digit fields are empty; welcome text and hint are displayed to the user. "Piano" instrument is selected Program appearance after startup

	
• User clicks piano keys.	• When a key is clicked, piano sound is played back even if another instrument is selected.
• User presses "Play melody" button.	• Program plays back a pre-recorded melody using a selected instrument (piano, guitar, cello or flute). • Clicking piano key after playback will not result in a new melody. Instead, new notes will be appended to the existing melody.
• User presses "Clear melody" button.	• A short melody deletion signal is played back. • Pressing "Play melody" when recorded melody has been deleted will not do any playback.

Stage 2. *Checking the student's knowledge level.*

Note. The teacher should use this section for assessing the overall knowledge of the group instead of public personal assessment of each student.

Each student must complete the undertaken project and test its results — this is *mandatory comprehension level*. After that, questions and problems are offered to each student. This is required to evaluate their knowledge depth.

Checking the student's knowledge level		Comprehension
Your synthesizer can play back a melody using various musical instruments. When you start the program, a welcome melody is played back on the piano. What happens if you select guitar and press "Play melody" after welcome?	Nothing, because welcome is an independent music file which is not linked to sound list.	Good
I want to record my favourite song and hear what it sounds like on different musical instruments. How can I do it using "Synthesizer" program?	You need to: enter the melody by pressing piano keys, select the desired musical instrument, and listen to the melody.	Good
I recorded a melody, then deleted it by pressing "Clear melody". After that, I changed my mind. How can I undo melody deletion?	There is no way to do this, because melody[] list has already been cleared. To undo melody deletion, you need to change the program so that the deleted file be copied in a backup list, and melody[] list be cleared.	Excellent



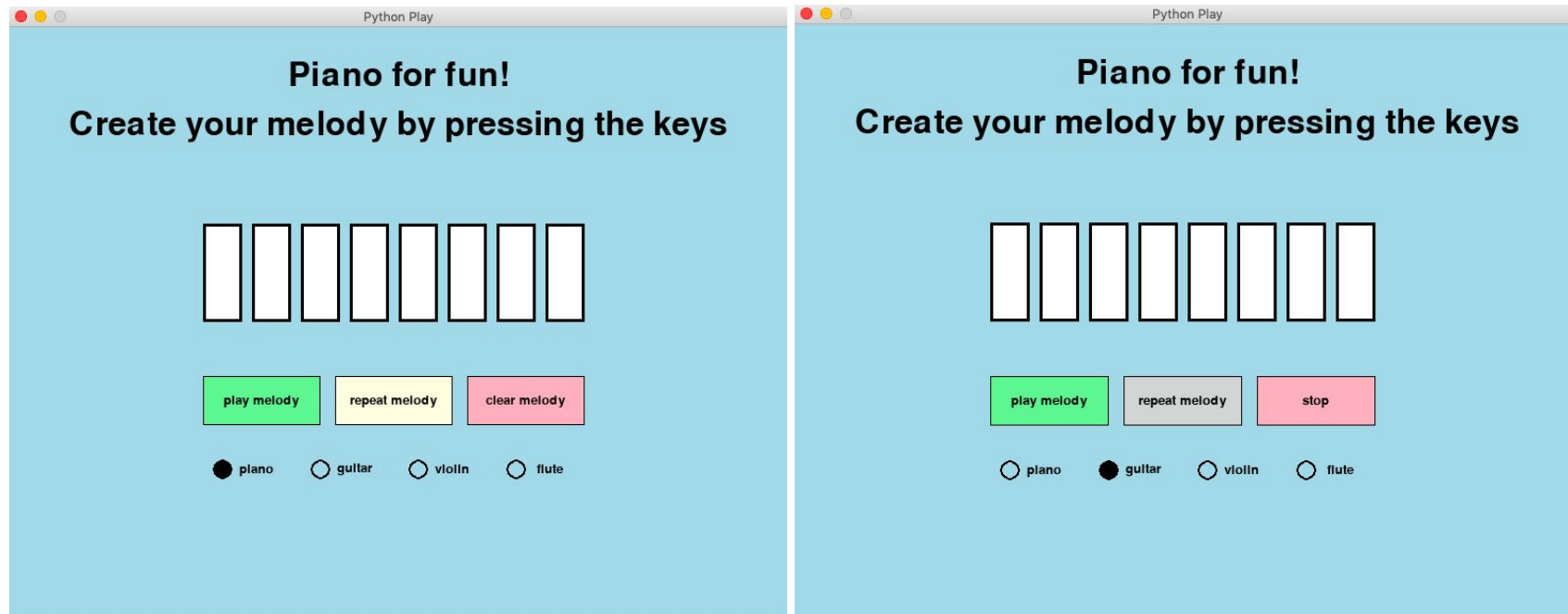
Part 6. Additional tasks for project improvement

Consider offering additional tasks to the students who want to improve their projects.

Task 1. Let us make the program more similar to a professional synthesizer. We shall add "Repeat melody" button which will play back the melody on selected instrument until user stops it manually.

When "Repeat melody" button is pressed, "Clear melody" button must change to "Stop". When user presses "Stop", they interrupt looping melody playback, while "Stop" changes back to "Clear melody".

Solution 1.



To implement this idea, a new global variable "ready" needs to be introduced to determine if the program is ready to work (True) or not (False). For looping playback of a recorded melody, it is convenient to use while().

```
import pygame
import play

instrument = 0
ready = False

# UI elements, selectors and sound loading are implemented in a similar manner
```

```

#...

@key_clear_melody.when_clicked
def clear():
    global ready
    if ready:
        melody.clear()
        sound_clear_melody.play()
    else:
        ready = True

@key_play_melody.when_clicked
# play back a recorded melody
async def play_m():
    global ready
    global instrument
    if ready:
        ready = False
        for i in range(len(melody)):
            await play.timer(seconds=0.5)
            sounds[instrument][melody[i]].play() # play a note of required instrument ("list of lists")
        ready = True

@key_repeat_melody.when_clicked
async def repeat_m():
    if ready:
        key_repeat_melody.color = 'light grey'
        key_clear_melody.color = 'light red'
        kcm.words = 'stop'
        ready = False
        global instrument
        i = 0
        while not ready:
            sounds[instrument][melody[i]].play()
            await play.timer(seconds=0.5)

```



```

        i += 1
        if len(melody) == i:
            await play.timer(seconds=0.5) # a pause at the end of a melodic line
            i = 0
    key_repeat_melody.color = 'light yellow'
    key_clear_melody.color = 'light pink'
    kcm.words = 'clear melody'

@play.repeat_forever
async def play_piano():
    global ready
    for i in range(len(keys)):
        if keys[i].is_clicked and ready:
            keys[i].color = 'light grey'
            sounds[0][i].play() # let keys always sound like piano
            await play.timer(seconds=0.3)
            keys[i].color = 'white'
            melody.append(i)

play.start_program()

```

Task 2. Let us help people who want to restore a deleted melody. For this purpose, the deleted melody needs to be copied and placed into the backup list (for example, `del_melody[]`). "Recover melody" button is required to empower a user to restore a deleted melody. When this button is pressed, deleted melody is restored into `melody[]`.

Dear developer, you have been assigned an order for **SYNTHESIZER** program.

Synthesizer must be able to:

- Greet user with a text background and a welcome melody.
- When keys are pressed, play back notes on piano.
- When one of four musical instruments is selected (guitar, cello, piano and flute), and Play Melody is pressed, the program plays a recorded melody on the selected instrument.

It is recommended to implement this program in Python **over 2 lessons**.

Testing is a critical stage of software development. It consists of 3 steps:

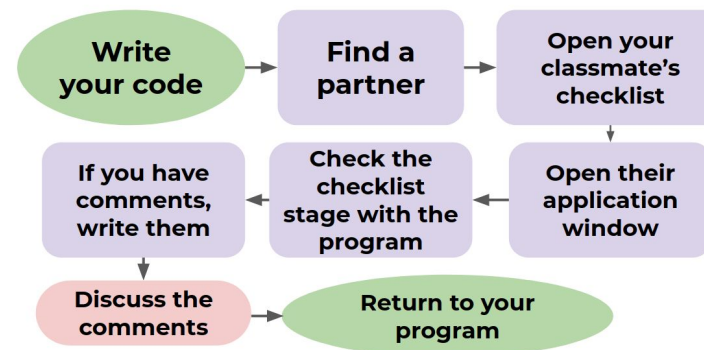
- *Independent* program operability check. For example, you need to record a melody (by pressing piano keys), switch instrument and listen to the melody in new sounding.
- *A developer peer review* performed in accordance with the following algorithm:

Project stages

- 📁 Learn about the idea/Generate the idea
- 🛠️ Get the tools ready (pictures, theory)
- 📅 Allocate project subtasks -> Project outline
- 🔄 Iterative work on the project: Program -> review (test) -> modify
- 📞 Delivery of the project to the customer



Code review algorithm



After successful testing, the project can be presented to the customer. This is the final stage of the project testing.